

ICMP Blind Connection-Reset & Blind Throughput Reduction Attack Against TCP

Final Implementation & Evaluation Report
CSE 406 — Computer Security Sessional

Anika Morshed (2105068)
Diganta Saha Tirtha (2105081)
Section: B1

September 18, 2026

Contents

1	Introduction	2
2	Attack 1: ICMP Blind Connection-Reset	2
2.1	Attack Process	2
2.2	Expected Outcome	3
2.3	Actual Outcome	3
2.4	Discrepancy Analysis: Why the Outcome Differed	3
2.5	Assumptions	3
3	Attack 2: ICMP Blind PMTU Throughput Reduction	4
3.1	Attack Process	4
3.2	Expected Outcome	4
3.3	Actual Outcome	5
3.4	Discrepancy Analysis: Why the Outcome Differed (Partially)	5
3.5	Assumptions	5
4	Defense Mechanisms	6
4.1	Sequence-Number Validation (Design Report Defense 1)	6
4.2	Ingress Filtering / BCP 38 (Design Report Defense 2)	6
4.3	PMTU Cache Aging / PLPMTUD (Design Report's Implementation Plan, P7)	7
4.4	Summary: Design-Report Defenses vs. Evaluated Reality	7
5	Consolidated Assumptions	8
6	Conclusion	9

1 Introduction

Both attacks exploit the same structural property of ICMP error handling: an ICMP error message embeds a copy of the original packet's IP and TCP headers (RFC 792) so the receiving host can match the error to one of its own connections. Because this embedded header is attacker-controlled once source-address spoofing is possible, a forged ICMP message that mimics a real connection's headers can influence a TCP stack's behavior without the attacker ever being on-path for that connection.

- **Attack 1** spoofs **ICMP Type 3, Code 3** (Port Unreachable) to trigger an RFC 1122 *hard error*, which a vulnerable stack uses to abort an established connection immediately.
- **Attack 2** spoofs **ICMP Type 3, Code 4** (Fragmentation Needed, DF Set) with a small advertised Next-Hop MTU, to shrink a bulk sender's Path MTU (and therefore its MSS) and degrade throughput.

Both were implemented and tested in a controlled lab of three Linux network namespaces (`client` 10.0.0.1, `server` 10.0.0.2, `attacker` 10.0.0.3) joined by a software bridge, as specified in the design report and built by `setup_topology.sh`.

2 Attack 1: ICMP Blind Connection-Reset

2.1 Attack Process

1. A client and server hold an ordinary, idle TCP connection open (via `ncat`).
2. The attacker passively sniffs the bridge (via traffic mirroring configured in `setup_topology.sh`) to observe the server's own outgoing segment and recover the client's ephemeral port and a sequence number known to be inside the server's current send window (`attacks/build_packets.py::capture`).
3. The attacker crafts a spoofed ICMP Type 3/Code 3 packet (`attacks/build_packets.py::build_reset_packet`) and injects it with Scapy's `send()` (`attacks/inject_reset.py`).
4. The server's kernel receives the ICMP, matches it against the live socket via the embedded header, and — if it treats the error as fatal — aborts the connection.

Packet structure used (corrected). The design report's original field table specified the embedded header as `src=Client`, `dst=Server`. Empirically, this direction never matched any socket on the server, because ICMP errors are delivered to the *original sender* of the embedded packet — the server has no socket where it is the destination of its own traffic. The corrected, working structure is:

Layer	Field	Value
Outer IP	Source	10.0.0.2 (spoofed – impersonates the server)
Outer IP	Destination	10.0.0.2 (the real server; the target)
ICMP	Type / Code	3 / 3 (Port Unreachable)
Embedded IP	Source → Destination	10.0.0.2 → 10.0.0.1 (server → client)
Embedded TCP	Source port → Dest. port	server's port → client's ephemeral port
Embedded TCP	Sequence number	sniffed from the server's own live outgoing segment

2.2 Expected Outcome

Per the design report (Section 5.1), a successful attack should produce:

1. The spoofed ICMP visible in the server's own `tcpdump` capture.
2. The embedded sequence number falling inside the server's receive window.
3. The server's TCP stack aborting the connection (RFC 1122 hard-error behavior).
4. `ss -tn` on the server showing the socket gone.
5. The client-side `ncat` process reporting a broken pipe / reset.

2.3 Actual Outcome

Items 1 and 2 were confirmed on every run: the spoofed packet reliably arrives and reliably passes the in-window check. **Items 3–5 did not occur against a realistic target.** Against a connection held with plain `ncat`, the same, correctly-formed packet was delivered but the connection **survived** – it remained `ESTAB` in `ss -tn` with no observable effect.

Scenario	ICMP delivered / in-window?	Result
Plain <code>ncat</code> (realistic target)	Yes	Survived (still <code>ESTAB</code>)
<code>recvrr_server.py</code> (<code>IP_RECVERR</code> set)	Yes	Reset (socket left <code>ESTAB</code> , typically)

To rule out a defect in the injected packet itself, a second target, `experiments/recvrr_server.py`, was built: an otherwise-identical TCP server that explicitly sets the `IP_RECVERR` socket option. Against this target the identical attack **did** reliably reset the connection (`ConnectionRefusedError` raised on the server, socket observed leaving `ESTAB`), confirming the packet's direction, sequence number, and checksums were all correctly formed.

2.4 Discrepancy Analysis: Why the Outcome Differed

Reading the Linux kernel source (`tcp_v4_err()` in `net/ipv4/tcp_ipv4.c`) explains the discrepancy. For a socket that is already in the `ESTABLISHED` state, Linux only treats this ICMP as an immediately-fatal RFC 1122 hard error if the owning application has explicitly opted in via the `IP_RECVERR` socket option; otherwise the kernel silently downgrades it to a non-fatal `sk_err_soft` and the connection is left untouched. Ordinary tools such as `ncat` do not set this option, so they are immune to this attack in its simplest form, while an application that does set it (a legitimate, documented use case for applications that want richer error reporting) remains vulnerable exactly as the design predicted.

This is a real, previously undocumented (in the design report) kernel defense, additional to the two the design report already lists in Section 6, and is reported as such in Section 5 of this report.

2.5 Assumptions

- The attacker can observe the live connection's traffic (via mirroring in this lab; in a real network, via some other on-path vantage point, a shared broadcast segment, or coarse inference/guessing of the sequence number).
- Ingress/egress source-address filtering is *not* enforced anywhere between the attacker and the target (`rp_filter` is explicitly disabled in the lab to permit this).

- The targeted application either does not rely on the specific `IP_RECVERR` behavior described above, or the report explicitly evaluates the case where it does (the positive control).
- The embedded sequence number remains inside the receiver's current window at the moment the forged packet is processed – trivially true here because the target connection is deliberately idle, so the window never moves.

3 Attack 2: ICMP Blind PMTU Throughput Reduction

3.1 Attack Process

1. A client sends a sustained bulk transfer to the server via `iperf3`.
2. The attacker sniffs the client's own outgoing segments to recover a live sequence number (`attacks/build_packets.py::capture_live_state()`).
3. The attacker crafts a spoofed ICMP Type 3/Code 4 packet with a small advertised Next-Hop MTU (576 bytes) and *sprays* 300 candidate packets, spaced 50,000 bytes apart in sequence space (`attacks/inject_pmtu.py`), repeating this every few seconds for the duration of the attack.
4. The client's kernel accepts the ICMP, updates its Path-MTU cache for the server, and shrinks its MSS accordingly.

Packet structure used. Unlike Attack 1, the design report's direction for this attack was already correct, because the client is both the connection's sender *and* the intended target (MSS is a per-sender cache):

Layer	Field	Value
Outer IP	Source	10.0.0.1 (spoofed)
Outer IP	Destination	10.0.0.1 (the real client; the target, as sender)
ICMP	Type / Code / Next-Hop MTU	3 / 4 / 576
Embedded IP	Source → Destination	10.0.0.1 → 10.0.0.2 (client → server)
Embedded TCP	Source port → Dest. port	client's ephemeral port → 5201
Embedded TCP	Sequence number	sniffed from the client's own live outgoing traffic

3.2 Expected Outcome

Per the design report (Section 5.2):

1. The spoofed ICMP visible in the client's capture.
2. `ip route get` showing `mtu 576` (PMTU cache updated).
3. TCP segment sizes dropping from ~ 1460 bytes to $\leq 576 - 40 = 536$ bytes.
4. A measurable decrease in `iperf3` throughput.

3.3 Actual Outcome

All four criteria were confirmed, across multiple independent runs. Representative measured values:

Metric	Before injection	After injection
<code>ss -tin (mss/pmtu)</code>	<code>mss:1448 pmtu:1500</code>	<code>mss:524 pmtu:576</code>
<code>ip route get 10.0.0.2</code>	<code>no cached MTU</code>	<code>cache expires <N>sec mtu 576</code>
Avg. TCP segment size	1448 bytes	531–542 bytes
<code>iperf3</code> throughput (run A)	95.21 Mbps	89.03 Mbps (−6.5%)
<code>iperf3</code> throughput (run B)	92.83 Mbps	87.57 Mbps (−5.7%)
ICMP packets observed	—	exactly (spray rounds $\times$ 300)

The ICMP-packet count matching *exactly* (spray rounds $\times$ 300) across every run is an internal consistency check confirming every sprayed candidate was actually delivered, not merely that some non-deterministic subset landed.

3.4 Discrepancy Analysis: Why the Outcome Differed (Partially)

The four qualitative criteria all held, but two aspects needed correction from a naive first implementation, both already anticipated in the design report’s phrasing but underestimated in magnitude:

- **A single sequence-number guess reliably fails.** Unlike Attack 1’s idle target, this connection is an active bulk transfer, so the kernel’s in-window check (`between(seq, snd_una, snd_nxt)`) – which applies to *every* ICMP type/code, not only the reset case – must be satisfied against a window that is only `cwnd*mss` wide (measured at $\sim$ 100–150 KB) and sliding forward at the full transfer rate. By the time one sniffed value passes through `tcpdump`, Python, and Scapy’s `send()`, it is almost always already acknowledged and out of window (confirmed via the `TcpExt:OutOfWindowIcmps` counter in `/proc/net/netstat`). Spraying 300 candidates across $\sim$ 15 MB of sequence space, in steps smaller than the in-flight window, is what reliably lands at least one packet inside the live window. This matches the design report’s own (plural) wording, “sampled seq numbers,” and the technique used in Watson’s original blind TCP injection research.
- **The throughput drop is real but modest (5–7%), not dramatic,** because the lab’s client–server link is byte-rate-limited to 100 Mbit (`tc tbf`) for realism. At a fixed byte rate, a smaller MSS mainly shifts more of every rate-capped packet toward the fixed 40-byte IP+TCP header instead of payload ($\sim$ 97% payload efficiency at MSS 1448 vs. $\sim$ 93% at MSS 536) – a real, explainable effect of the lab’s shaping choice, not evidence the attack is weak.

3.5 Assumptions

- The attacker can observe the live connection’s own outgoing traffic to recover a plausible base sequence number.
- No ingress/egress source-address filtering exists between the attacker and the target.

- The attacker can send the spray fast enough, and repeat it often enough, to keep at least one candidate inside the constantly shifting in-window range – a persistent-attacker assumption, explicitly acknowledged in §4 below when discussing why a self-healing defense can be undermined by re-spraying.
- The target’s application does not itself detect or route around the reduced MSS in a way that this report has not modeled (e.g. redundant multipath).

4 Defense Mechanisms

Two defenses named in the design report (Section 6) were implemented and evaluated, plus one already-listed defense that was confirmed as an emergent property of the attacks themselves rather than a separately built mechanism.

4.1 Sequence-Number Validation (Design Report Defense 1)

Mechanism. Modern Linux kernels reject any ICMP error whose embedded sequence number falls outside the target connection’s current window, regardless of whether the source address is genuine.

Expected outcome. Blind sequence-number guessing becomes substantially harder against an active connection.

Actual outcome. **As expected.** This defense was not implemented separately – it is already active, by default, in the lab’s stock Linux kernel, and its effect is precisely what forced Attack 2’s design away from a single guess toward the 300-candidate spray described in Section 3.3. Its presence is confirmed indirectly but unambiguously by the `TcpExt:OutOfWindowIcmps` counter incrementing on single-shot attempts.

4.2 Ingress Filtering / BCP 38 (Design Report Defense 2)

Mechanism. A border router or switch port refuses to forward any packet whose source address does not belong to the network segment it physically arrived from. Simulated in this lab with a `tc` filter on the bridge port the attacker’s traffic physically arrives on (`veth-at-br`), permitting only `src=10.0.0.3` (the attacker’s genuine address) and dropping everything else:

```
tc qdisc add dev veth-at-br handle ffff: ingress
tc filter add dev veth-at-br parent ffff: protocol ip prio 1 u32 \
    match ip src 10.0.0.3/32 action ok
tc filter add dev veth-at-br parent ffff: protocol ip prio 2 u32 \
    match u32 0 0 action drop
```

Expected outcome. Both attacks should be blocked completely, regardless of the target’s own TCP-stack-specific behavior, since the spoofed packet is stopped before it ever reaches the bridge.

Actual outcome. **As expected.** Re-running both attacks with the filter active produced no ICMP at either target and no change to connection state, MSS, PMTU, segment size, or throughput in either attack. Critically, the `recvrr_server.py` scenario – which *always* resets without this defense – **survived** with the filter active, isolating that the defense operates at the network layer and is independent of the target application’s socket options. This was the single most conclusive and reliable defense evaluated.

Scenario (filter active)	ICMP delivered?	Result
Attack 1, plain ncat	No	Survived (unchanged)
Attack 1, <code>recvrr_server.py</code>	No	Survived (previously reset without the filter)
Attack 2, iperf3 bulk transfer	No	<code>mss/pmtu</code> unchanged; throughput unaffected

4.3 PMTU Cache Aging / PLPMTUD (Design Report's Implementation Plan, P7)

Mechanism (as specified). The design report's Implementation Plan instructs: "Enable `tcp_mtu_probing=2`. Repeat P4 and P5. Record whether the kernel rejects the forged ICMP messages." The intent is that Linux's Packetization-Layer PMTU Discovery (PLPMTUD) would cause the client to re-verify its true path MTU and recover from the forged value on its own.

Expected outcome. A single spoofed PMTU packet degrades the connection's MSS momentarily, then the client's own kernel-level recovery restores `mss/pmtu` to their original values within a short window, without any further attacker or network-layer intervention.

Actual outcome. **Differed from expectation.** Two successive, independently testable hypotheses about *how* this recovery should occur were each implemented and tested, and **neither produced the expected recovery** within the tested window:

1. **First hypothesis:** `net.ipv4.tcp_mtu_probing=2` together with a shortened `net.ipv4.tcp_probe_interval` (5s) would cause active re-probing and recovery. *Result:* `mss/pmtu` remained pinned at the forged 524/576 for over 20 continuous seconds. On inspection, `tcp_mtu_probing` governs detection of ICMP *black holes* (a router that silently drops oversized packets and never replies at all) and has no role in re-verifying a PMTU value the kernel already accepted from an ICMP message it did receive.
2. **Second hypothesis:** the route-cache PMTU *exception* created when the ICMP lands (visible as `cache expires <N>sec mtu 576` in `ip route get`) ages out via `net.ipv4.route.mtu_expires` (600s by default); this was shortened to 8s to make recovery observable within a reasonable test window. *Result:* even after confirming the exception was created with the shortened timer, `mss/pmtu` were still pinned at 524/576 at every checkpoint out to $t+40$ s.

Status at the time of writing. This defense is **Unresolved / under investigation**. A diagnostic step was added (checking whether the route-table exception itself has actually been evicted by $t+40$ s, then forcibly flushing the route cache to determine whether an already-established, continuously busy socket needs an explicit external trigger to re-resolve its cached PMTU) but had not yet produced a confirmed root cause at submission time. The most plausible working explanation is that Linux caches a socket's last-known PMTU (`icsk.pmtu_cookie`) and only re-validates it opportunistically — not automatically upon a route-table exception's expiry elsewhere — so a continuously active socket may never receive that trigger on its own.

4.4 Summary: Design-Report Defenses vs. Evaluated Reality

Defense	Verdict	Notes
Sequence-number validation	As expected	Inherent to the stock kernel; confirmed indirectly via the need for Attack 2's 300-packet spray.
Ingress filtering (BCP 38)	As expected	Blocks both attacks completely; independent of target application behavior; the single most reliable defense tested.
<code>tcp_mtu_probing=2</code> (P7, as literally specified)	Differed from expectation	Governs ICMP black-hole detection, not recovery from an <i>accepted</i> forged ICMP; produced no observed recovery.
PMTU route-cache aging (follow-up hypothesis, not in the original design report)	Unresolved / under investigation	Shortened <code>mtu_expires</code> alone also failed to produce recovery within 40s; root cause still being isolated via a route-cache-flush diagnostic.
<code>IP_RECVERR</code> gating (discovered during Attack 1 testing, not in the original design report)	As expected	Not a designed defense, but a real, already-built-in Linux behavior that defeats Attack 1 against any application that does not explicitly request richer ICMP error delivery.

5 Consolidated Assumptions

The following assumptions are shared across both attacks and are necessary preconditions for either to succeed at all:

1. **No source-address filtering** exists anywhere between the attacker and the target – disabled explicitly in this lab (`rp_filter=0`) to permit spoofing, and shown in Section 4.2 to fully neutralize both attacks when reintroduced.
2. **The attacker can observe enough of the live connection** (source/destination ports and an in-window sequence number) to construct a convincing embedded header – provided here by passive traffic mirroring, but achievable in principle by other means (e.g. a shared broadcast segment, or brute-force sequence guessing at a cost proportional to the window size).
3. **The target's TCP stack processes the embedded header using the packet's actual direction semantics** – i.e. ICMP errors are delivered to, and matched against, the original sender of the embedded packet. This governed which side (server vs. client) each attack had to impersonate, and was the single correction required to the design report's original packet tables.
4. **For Attack 1 specifically:** the target application either sets `IP_RECVERR` (uncommon, e.g. specialized error reporting) or the report separately evaluates that narrower case as a positive control, since the attack does not succeed against a typical established connection without it.
5. **For Attack 2 specifically:** the attacker must be able to repeat the spray persistently against an undefended target, since (regardless of which recovery mechanism eventually turns out to govern it) a defended or self-healing target can undo a single-shot attack given enough time.

6 Conclusion

Both attacks were successfully implemented and demonstrated the core mechanism described in the design report: a spoofed ICMP error, carrying a forged embedded header, can influence a real TCP connection's behavior without the attacker being on-path. Attack 2 (PMTU reduction) matched its design-time expectations closely once corrected for the in-window sequence constraint. Attack 1 (connection reset) required a larger correction: it does *not* reliably work against a realistic target, and its true precondition (`IP_RECVERR`) is narrower than the design report assumed – a materially different, and arguably more interesting, result than a simple confirmation would have been.

Of the two evaluated defenses, ingress filtering proved to be a complete and reliable countermeasure against both attacks, while the kernel self-healing mechanism the design report's Implementation Plan specified (`tcp_mtu_probing=2`) did not produce its expected effect under either of two tested hypotheses for why it should. Reporting this negative result, rather than only reporting successes, is treated in this report as itself a legitimate and important finding: not every plausible, textbook-named defense behaves as its name suggests once it is actually measured.

References

- [1] J. Postel, *Internet Control Message Protocol*, RFC 792, Sept. 1981.
- [2] R. Braden (Ed.), *Requirements for Internet Hosts — Communication Layers*, RFC 1122, Oct. 1989.
- [3] J. Mogul and S. Deering, *Path MTU Discovery*, RFC 1191, Nov. 1990.
- [4] P. Watson, *Slipping in the Window: TCP Reset Attacks*, CanSecWest, 2004.
- [5] A. Ramaiah, R. Stewart, M. Dalal, *Improving TCP's Robustness to Blind In-Window Attacks*, RFC 5961, Aug. 2010.
- [6] P. Ferguson and D. Senie, *Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing*, BCP 38 / RFC 2827, May 2000.
- [7] F. Gont, *Deprecation of ICMP Source Quench Messages*, RFC 6633, May 2012.